Sideline — Complete Feature Specification

> **Sports Team Management System — Discord-First Architecture** Primary sport: Ultimate Frisbee | Multi-sport compatible https://sideline.cz Target: Bachelor's Thesis Project Developer: Solo | Timeline: 12+ months

---

Scope Summary

Sideline is a **Discord-first team management platform** that replaces the spreadsheets and group chat chaos amateur sports teams rely on today. Sideline is the **single source of truth** for team data — roster, roles, permissions, events, attendance, finances, and training team generation — with Discord serving as the primary player-facing interface and the web app serving as the admin/management interface.

Target user: One ultimate frisbee team (~20–30 players) **Auth:** Discord OAuth 2.0 **Languages:** Czech 🇨🇿 + English 🇬🇧 (admin-editable translations) **Architecture principle:** Sideline owns the data, Discord reflects it.

Core Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    SIDELINE (Backend)                    │
│                 Single source of truth for:              │
│          roster, roles, permissions, events,             │
│          attendance, finances, Elo ratings,              │
│          makáníčko, channels, translations               │
├──────────────────────────┬───────────────────────────────┤
│        Web App           │        Discord Bot            │
│      (Admin UI)          │     (Player-facing UI)        │
│                          │                               │
│ • Manage all             │ • Slash commands              │
│   settings               │ • Button interactions         │
│ • CMS for                │ • RSVP via buttons            │
│   translations           │ • Role sync                   │
│ • Channel                │ • Channel management          │
│   management             │ • Event notifications         │
│ • Finances               │ • Email forwarding            │
│ • Team                   │ • Team generation             │
│   generator              │ • Makáníčko logging           │
│ • Reports                │ • Achievement roles           │
└──────────────────────────┴───────────────────────────────┘
```

Version Strategy

Reasoning

v1 (Thesis MVP) — The features that make Sideline usable by your team day-to-day, replacing spreadsheets and Discord chaos. This is what you demo, what your team uses, and what your thesis evaluates. Must be **rock solid**.

v2 (Thesis Stretch / Post-Launch) — Features that add significant value but aren't required for basic daily operation. Some may make it into the thesis if time allows. These are designed and documented in the thesis as "planned features" regardless.

Key principle: Every v1 feature must be fully polished — working on both web and Discord, tested, documented. It's better to have 8 great features than 14 half-working ones.

Version Assignment

#	Feature	Version	Rationale
1	Discord OAuth + onboarding	**v1**	Foundation — nothing works without auth
2	Player roster & profiles	**v1**	Core data — everything depends on this
3	Age-based role management	**v1**	Directly requested by team, simple to build on top of profiles
4	Centralized permissions & roles	**v1**	Architectural foundation — Discord sync, channels, and everything else depends on this
5	Training types & coach binding	**v1**	Core permission scoping — coaches tied to their training type
6	Discord role sync	**v1**	Inseparable from centralized permissions
6	Custom captain-helper subgroups	**v1**	Team specifically needs this, creates Discord channels
7	Event scheduling (recurring + single)	**v1**	Core daily use — replaces spreadsheet scheduling
8	RSVP with deadlines + custom messages	**v1**	Core daily use — the #1 pain point being solved
9	RSVP via Discord buttons	**v1**	Players live in Discord — this is the primary interaction
10	RSVP reminders	**v1**	Essential for RSVP to actually work
11	Minimum player threshold warnings	**v1**	Small feature, big value for captains

#	Feature	Version	Rationale
12	Discord event notifications	**v1**	Events are useless if nobody sees them
13	Personal calendar subscription (iCal)	**v1**	Low effort, high convenience — players subscribe once
14	Dashboard (web)	**v1**	Landing page after login, aggregates everything
15	Responsive / mobile-first design	**v1**	Non-negotiable for a web app in 2026
16	Internationalization (CZ + EN)	**v1**	Czech team, English thesis — needed from day one
17	Makáníčko activity tracking	**v1**	High engagement feature, team wants it, moderate complexity
18	Makáníčko leaderboard	**v1**	Simple aggregation, makes tracking worthwhile
19	Makáníčko achievements + Discord roles	**v2**	Depends on tracker being stable first; achievement threshold logic + auto-role-granting adds complexity
20	Makáníčko weekly reports	**v2**	Nice-to-have automation, not critical for tracker to be useful
21	Makáníčko custom activities (admin-defined)	**v2**	Predefined activities are enough for v1
22	Training team generator	**v2**	Thesis centerpiece algorithm but depends on Elo data — needs training results history first
23	Elo rating system	**v2**	Paired with team generator — both ship together
24	Training game result logging	**v2**	Paired with Elo — feeds the rating system
25	Finances: fee management + payment tracking	**v2**	Team needs it but it's a self-contained module; can be added without touching v1 features
26	Finances: basic expense tracking	**v2**	Ships with fee management as a complete finance module
27	Finances: payment reminders (Discord + iCal)	**v2**	Ships with finance module
28	Tournament attendance bot	**v2**	Specialized flow, tournaments are seasonal
29	Discord channel CRUD from web app	**v2**	Useful but admins can create channels in Discord directly for now
30	Translation CMS (admin-editable)	**v2**	i18n architecture ships in v1 (developer-managed JSON files); admin CMS is a convenience layer
31	Email forwarding + AI summarization	**v2**	Highest infrastructure complexity; email ingestion + AI pipeline + approval flow is a project in itself

#	Feature	Version	Rationale
32	Coach approval flow for emails	**v2**	Ships with email forwarding
33	Achievement configuration UI (admin)	**v2**	Depends on achievements being stable

v1 Features — Detailed Specification

1. Authentication & Onboarding

- Discord OAuth 2.0 login (sole auth method)
- Discord identity auto-linked to Sideline profile (avatar, username)
- Team join via shareable invite link
- First-time profile completion: name, birth year, jersey number, position(s), gender, proficiency level
- Onboarding detects Discord server membership

2. Player Roster & Profiles

- Name, photo (from Discord), jersey number, position(s)
- Birth year, gender, proficiency level
- Contact info, player status: active / inactive
- Discord account link (visible to admins)
- Roster overview with search and filtering
- Admin can add/remove/deactivate players

3. Age-Based Role Management

- Fixed age thresholds (e.g., U20, U24) configured by admin
- When a player ages out → role automatically removed
- Role changes sync to Discord automatically
- Admin configures thresholds and role mappings

4. Centralized Permissions & Roles

4.1 Role System

- **Built-in roles:** Admin, Captain, Player
- **Custom roles:** Admins create additional roles (e.g., Junioři, Trenér)
- **Captain-helper subgroups:** Fully custom — admins create, name, manage freely

    - Examples: trainers, socials, makáníčko, newbie-coaches, zamyšlení
    - Each subgroup gets its own permissions and associated Discord channel

4.2 Permission Model

- Granular permissions defined per role in Sideline
- Each frontend maps Sideline permissions to its own access control:
    - **Web app:** UI elements shown/hidden based on role
    - **Discord:** Roles and channel permissions synced from Sideline

4.3 Discord Role Sync

- Sideline role changes → Discord roles updated in real-time
- New custom roles → Discord role auto-created
- Role removal → Discord role auto-removed

5. Training Types & Coaching

5.1 Training Type System

- **Configurable training types** — admins define types (e.g., "Main Training", "Strength Training", "Conditioning")
- Each training type has:
    - Name, description, emoji/icon
    - Its own **recurring schedule** (e.g., Main = Tue/Thu 18:00, Strength = Mon/Wed 17:00)
    - **Assigned coaches** — only these coaches can manage events of this type
- Admins can create, edit, and delete training types at any time

5.2 Coach–Training Type Binding

- A coach role is **always tied to a specific training type**
- "Strength Training Coach" and "Main Training Coach" are distinct roles
- A coach can only:
    - Create/edit/cancel events of their assigned training type
    - View RSVP details for their training type
    - Post to their training type's Discord channel
- A person can be coach for multiple training types (holds multiple coach roles)
- Admins and captains retain full access across all training types

6. Event Scheduling & Calendar

6.1 Event Types

- **Recurring trainings:** Each training type has its own recurrence and schedule, editable per-instance or entire series
- **Single events:** Training camps, team meetings, social events (not tied to a training type)
- Event details: date, time, duration, location (address + map link), description, **training type** (for trainings)

6.2 Calendar

- Calendar view (month/week) on web app
- Filterable by training type
- **Personal calendar subscription** (iCal/webcal feed) with RSVP'd events
- i18n-aware date/time formatting

6.3 Event Management

- Create, edit, cancel events:
  - **Trainings:** Only by assigned coach for that type, or admin/captain
  - **Single events:** By admin/captain
- Edit single occurrence or entire recurring series
- Discord bot posts event to configured channel (can be per training type)
- Event updates and cancellations update the Discord message

6. RSVP & Attendance

6.1 RSVP System

- RSVP options: ✅ Yes / ❌ No / ❓ Maybe
- **Custom message** on RSVP (e.g., "arriving 15 min late")
- RSVP deadline per event
- RSVP via **web app** (button on event/dashboard) and **Discord** (interactive buttons)
- Real-time bidirectional sync between web and Discord

6.2 Reminders & Warnings

- Automated RSVP reminders via Discord before deadline
- Minimum player threshold — visual warning when headcount is low

- Non-responders list visible to captains/admins

7. Makáníčko Tracker (Activity Tracking + Leaderboard)

7.1 Activity Tracking

Auto-tracked (no player action needed):

- Training attendance (from RSVP/check-in)
- Injury status (logged by coach/admin)

Player-logged (predefined types):

- 🏋️ Gym workouts (strength training)
- 🏃 Running / cardio
- 🧘 Stretching / mobility

7.2 Logging Interface

- **Discord:** `/makanicko log` → select activity type, optional duration/note
- **Web app:** Quick-log button on dashboard
- Each log: activity type, date (defaults today), optional duration, optional note
- Players can view and edit their own history

7.3 Leaderboard

- **Public** — visible to entire team
- Sortable by: total activities, weekly activities, current streak, longest streak
- Filterable by activity type
- Displayed on web dashboard
- Discord command: `/makanicko leaderboard`

8. Discord Bot

> All features use **slash commands and button/select interactions** only.

8.1 Event & RSVP

- `/event create` — create event (captain+)
- `/event list` — show upcoming events
- Event messages with interactive RSVP buttons + custom message field
- Automated reminders before RSVP deadline
- Attendance summary posted before event

8.2 Makáníčko

- `/makanicko log` — log an activity
- `/makanicko stats` — view your stats and streaks
- `/makanicko leaderboard` — show team leaderboard

8.3 Role & Permission Sync

- Sideline role changes → Discord roles updated in real-time
- Age-based role removal synced automatically
- Channel access controlled by Sideline permissions

9. Dashboard (Web App)

- Personal dashboard after login
- Upcoming events (next 7 days)
- Events awaiting your RSVP
- Quick RSVP actions
- Makáníčko: your recent activity, current streak, leaderboard position
- Quick-log makáníčko activity

10. Internationalization

- Full Czech 🇨🇿 + English 🇬🇧 support
- Language switcher in web UI
- Discord bot responds in server's configured language
- All UI strings in developer-managed i18n files (JSON)
- i18n architecture ready for admin CMS in v2

11. Team Settings & Admin

- Team name, logo, description, sport
- Manage invite link (generate, regenerate, disable)
- Discord server connection settings
- Bot channel configuration
- Age threshold configuration
- Custom role and subgroup management

12. Responsive Design

- Mobile-first responsive web app
- PWA-ready structure (installable, app-like)
- Primary player interaction expected via Discord

---

v2 Features — Detailed Specification

13. Training Team Generator

13.1 Team Generation

- Admin/coach triggers generation for a training event
- Inputs: players from RSVP "Yes" list
- Mode: admin picks **team count** OR **team size**
- Algorithm balances teams by:
  - **Elo rating** (primary) — aims for equal average Elo per team
  - **Gender** — respects configurable ratio constraints
  - **Age** — distributes age groups evenly
  - **Proficiency level** — balances experience
- Coach can **manually adjust** (swap players) after generation
- Teams posted to Discord and displayed on web

13.2 Elo Rating System

- Base rating: 1000 for new players
- After training games, coach logs **which team won**
- Winning team players gain Elo; losing team players lose Elo
- Elo change considers expected outcome (team Elo difference)
- Rating history tracked over time
- Elo visible to coaches/admins
- New players: higher K-factor (faster calibration)

13.3 Training Game Result Logging

- Coach selects winning team per game
- Multiple games per training supported
- Results feed into Elo immediately

- Historical results stored

13.4 Discord Commands

- `/training generate` — generate teams
- `/training result` — log which team won

14. Finances & Payments

14.1 Fee Management

- Define membership/season fees with due dates
- Per-event fees (field rental share, tournament entry)
- Custom fee categories

14.2 Payment Tracking

- Status per player: paid / partial / overdue / waived
- Manual payment logging (bank transfer, cash)
- Payment history per player
- Bulk overview (who's paid, who hasn't)
- Payment reminders via Discord + iCal feed

14.3 Basic Expense Tracking

- Log expenses: amount, date, category, description
- Categories: fields, equipment, travel, tournament fees, other
- Income vs. expense overview
- Admin/treasurer dashboard with balance

14.4 Discord Commands

- `/finance status` — check your payment status
- Payment reminders via DM or channel

15. Makáníčko Achievements & Roles

15.1 Achievement System

- Unique badges with threshold conditions
- Predefined achievements with editable thresholds:
    - Examples: "10 gym sessions", "30-day streak", "100 total activities"

- Each achievement: name, description, emoji/icon, threshold, Discord role

15.2 Discord Role Integration

- Achievement earned → Discord role **automatically granted**
- Achievement announcement posted to team channel
- Roles persist once earned

16. Makáníčko Weekly Reports

- Auto-generated weekly summary per player
- Contents: activity count per type, streak status, new achievements
- Posted to Discord and viewable on web
- Team-wide aggregate for coaches

17. Makáníčko Custom Activities

- Admins define additional activity types
- Each: name, emoji, description
- Players can log custom activities same as predefined

18. Tournament Attendance Bot

- Interactive attendance tracking with Discord buttons
- Dedicated tournament flow (more advanced than regular RSVP)
- Visual attendance overview posted to Discord

19. Discord Channel Management (from Web App)

- **Create channels** from web → bot creates in Discord
- **Rename channels** from web → synced to Discord
- **Archive channels** from web → bot archives in Discord
- **Manage membership** via Sideline permissions
- Subgroup channels auto-created with subgroups

20. Translation CMS

- Admin interface to edit all UI translations
- Key-value editor: translation key → Czech / English value
- Changes take effect immediately (no redeployment)

- Fallback: missing translation → other language or key
- Export/import translations

21. Email Forwarding & AI Summarization

- Organizational emails auto-forwarded to processing pipeline
- AI generates summary
- **Coach approval flow:** summary shown to coach via Discord DM
- Coach approves → posted to team channel
- Coach rejects → original posted or discarded
- Configurable: monitored email address(es), target channel

User Roles & Permissions

Action	Admin	Captain	Coach (typed)	Subgroup Lead	Player
Manage team settings	✅	❌	❌	❌	❌
Manage roster (add/remove)	✅	❌	❌	❌	❌
Create/manage roles & subgroups	✅	❌	❌	❌	❌
Configure training types & assign coaches	✅	❌	❌	❌	❌
Configure Discord bot & channels	✅	❌	❌	❌	❌
Configure age thresholds	✅	❌	❌	❌	❌
Manage finances (v2)	✅	✅	❌	❌	❌
Manage achievements (v2)	✅	❌	❌	❌	❌
Configure email forwarding (v2)	✅	❌	❌	❌	❌
Approve email summaries (v2)	✅	✅	❌	❌	❌
Create/edit/cancel ANY event	✅	✅	❌	❌	❌
Create/edit/cancel OWN training type	✅	✅	✅	❌	❌
View RSVP for own training type	✅	✅	✅	❌	❌
Generate training teams (v2)	✅	✅	✅ (own type)	❌	❌
Log training game results (v2)	✅	✅	✅ (own type)	❌	❌

Action	Admin	Captain	Coach (typed)	Subgroup Lead	Player
Adjust generated teams (v2)	✅	✅	✅ (own type)	❌	❌
Manage their subgroup channel	✅	✅	❌	✅	❌
RSVP to events (+ custom msg)	✅	✅	✅	✅	✅
Log makáníčko activities	✅	✅	✅	✅	✅
View leaderboard	✅	✅	✅	✅	✅
View roster & calendar	✅	✅	✅	✅	✅
View own payment status (v2)	✅	✅	✅	✅	✅
Edit own profile	✅	✅	✅	✅	✅
Subscribe to personal calendar	✅	✅	✅	✅	✅

Core User Flows

v1 Flows

Flow 1: New Player Joins

1. Receives invite link from captain
2. Clicks link → Discord OAuth login
3. Profile auto-populated from Discord (name, avatar)
4. Completes profile: birth year, gender, jersey number, position, proficiency
5. System assigns roles based on age (e.g., Junioři if U20)
6. Discord roles synced automatically
7. Lands on dashboard → sees upcoming events

Flow 2: Captain Creates Weekly Training

1. Admin has configured training types: "Main Training" (coaches: Jan, Petra) and "Strength" (coach: Tomáš)
2. Jan (Main Training coach) creates recurring event: "Tuesday Main Training, 18:00–20:00"
3. Sets location, description, RSVP deadline (Monday 20:00)
4. Saves → all instances created in calendar under "Main Training"
5. Discord bot posts first occurrence to training channel

6. Players RSVP via Discord buttons (with optional custom message like "coming late")
7. Monday 12:00: bot sends reminder to non-responders
8. Monday 20:00: deadline passes, Jan sees final headcount + messages
9. Tomáš (Strength coach) cannot edit or cancel Jan's training — only his own Strength sessions

Flow 3: Admin Creates Captain-Helper Subgroup

1. Admin opens Sideline web → Subgroups section
2. Creates new subgroup: "Socials Team"
3. Assigns members from roster
4. Sideline creates corresponding Discord role + private channel
5. Members automatically get channel access

Flow 4: Player Ages Out of Junioři

1. System runs periodic age check
2. Player crosses configured age threshold
3. "Junioři" role automatically removed in Sideline
4. Discord role removed via bot sync
5. Admin notified of the role change

Flow 5: RSVP via Discord

1. Bot posts event with ✅ Yes / ❌ No / ❓ Maybe buttons
2. Player clicks a button → modal appears for optional custom message
3. RSVP syncs to Sideline in real-time
4. Player can change RSVP by clicking a different button
5. Updated count + messages visible on Discord and web

Flow 6: Makáníčko — Log Activity

1. Player does a gym workout
2. Opens Discord → `/makanicko log` → selects "Gym workout"
3. Optionally adds duration and note
4. Activity logged → streak counter increments
5. Leaderboard updated

v2 Flows

Flow 7: Training Team Generation & Results

1. Training day — 18 players RSVP'd "Yes"
2. Coach opens event → clicks "Generate Teams"
3. Selects: 3 teams of 6
4. Algorithm generates 3 balanced teams (Elo + gender + age + proficiency)
5. Coach reviews → swaps 2 players → confirms
6. Teams posted to Discord
7. After games: coach logs "Team A won Game 1", "Team C won Game 2"
8. Elo ratings recalculated for all 18 players

Flow 8: Fee Payment Tracking

1. Admin creates season fee: "Spring 2026 — 2000 CZK, due March 1"
2. All active players see fee on dashboard
3. Discord bot sends payment reminders
4. Reminder appears in player's iCal feed
5. Player pays via bank transfer → admin logs payment
6. Status updates to "paid"

Flow 9: Organizational Email Arrives

1. Email received at monitored address
2. AI generates summary
3. Summary sent to coach via Discord DM
4. Coach approves → posted to team channel

Flow 10: Player Earns Achievement

1. Player logs their 10th gym session
2. System evaluates achievement thresholds
3. "🏋️ Gym Rat" achievement unlocked
4. Discord role automatically granted
5. Announcement posted to team channel

Complexity Assessment

v1

Feature	Effort	Risk	Notes
Discord OAuth + profiles	Medium	Low	Well-documented API
Roster + roles	Medium	Low	Standard CRUD
Age-based role management	Low	Low	Date math + periodic check
Centralized permissions → Discord sync	**High**	**High**	Core architecture — must be designed first
Training types + coach permissions	Medium	Medium	Extends role system with type scoping
Custom subgroups + Discord channels	Medium	Medium	Depends on permission model
Event scheduling + recurring	**High**	Medium	Recurring event logic is complex
RSVP (web + Discord bidirectional)	**High**	Medium	Real-time sync needs careful design
RSVP reminders + warnings	Low	Low	Cron + threshold check
Makáníčko activity tracking	Medium	Low	CRUD + streak logic
Makáníčko leaderboard	Low	Low	Aggregation queries
iCal subscription	Low	Low	Standard iCal generation
Dashboard	Medium	Low	Aggregation of existing data
i18n (CZ + EN, file-based)	Medium	Low	Setup once, maintain as you build
Responsive / PWA	Medium	Low	CSS + manifest, ongoing effort

v2

Feature	Effort	Risk	Notes
Team generator + Elo	**High**	**Medium**	Optimization algorithm — thesis highlight
Training result logging	Low	Low	Simple W/L per game
Finances (fees + expenses)	Medium	Low	Self-contained CRUD module
Payment reminders	Low	Low	Extends existing reminder system
Achievements + Discord roles	Medium	Medium	Threshold engine + role sync

Feature	Effort	Risk	Notes
Weekly reports	Low	Low	Cron + templated messages
Custom activities	Low	Low	Extends existing tracker
Tournament attendance bot	Medium	Medium	Custom interaction flow
Discord channel CRUD	Medium	Medium	Discord API rate limits
Translation CMS	Medium	Low	Key-value UI + live reload
Email + AI summarization	**High**	**High**	Email ingestion + AI + approval flow

---

Recommended Build Order (v1)

A suggested sequence that respects dependencies:

1. **Project setup** — repo, CI/CD, database, deployment pipeline
2. **Discord OAuth + basic profiles** — get login working
3. **Centralized permission model** — design and build the role/permission engine
4. **Discord bot foundation** — bot connects, slash command framework, role sync
5. **Roster management** — CRUD on web, role assignment, Discord sync
6. **Age-based roles** — build on roster + permissions
7. **Custom subgroups** — build on roles + Discord channel creation
8. **i18n setup** — integrate early so all new UI is translatable
9. **Event scheduling** — recurring events, calendar view
10. **RSVP system** — web + Discord buttons + bidirectional sync
11. **RSVP reminders + warnings** — cron jobs, Discord notifications
12. **Makáníčko tracker** — activity logging, streak logic
13. **Makáníčko leaderboard** — aggregation, Discord command
14. **iCal subscription** — personal calendar feed
15. **Dashboard** — aggregate all data into home screen
16. **Polish** — responsive design, PWA, edge cases, testing